

# Análisis y Plan de Proyecto: Sistema de Gestión Académica (SGA-DO)

## 1. Análisis de la Crisis de DiplomadosOnline

La crisis actual de DiplomadosOnline es un problema clásico de escalabilidad no planificada. El éxito comercial y el crecimiento exponencial de la matrícula superaron por completo la infraestructura operativa de la academia. Actualmente, todo el peso de la gestión académica y administrativa —desde el registro de estudiantes y profesores hasta el control de notas— recae en un único archivo de Excel compartido.

Esta dependencia tecnológica ha desencadenado tres fallas críticas en la operación diaria:

- **Corrupción y pérdida de datos:** Al manipular un archivo plano con miles de registros de forma concurrente, el sistema es inestable. Los cierres inesperados se traducen directamente en la pérdida de horas de trabajo administrativo.
- **Vulnerabilidad en la integridad de la información:** La ausencia de un sistema de validación permite errores de tipeo, duplicidad de identificaciones (cédulas) y cruce de datos erróneos entre profesores y alumnos, ensuciando la base de datos.
- **Colapso en el proceso de certificación:** Al carecer de automatización, el personal debe calcular manualmente los promedios de cada estudiante basándose en reglas de negocio que varían según el tipo de programa (Curso, Diplomado o Bootcamp). Este cálculo manual fila por fila retrasa la emisión de certificados por semanas, afectando gravemente la experiencia del estudiante que ya culminó su formación.

## 2. Requerimientos del Producto Mínimo Viable (MVP)

Para resolver esta crisis operativa, el MVP del nuevo *Motor Core (Backend)* debe prescindir de funciones complejas de interfaz gráfica y centrarse estrictamente en la estabilidad, persistencia y automatización de las reglas de negocio a través de la consola. Los requerimientos indispensables son:

- **Persistencia de Datos en Texto Plano:** Migración del Excel a archivos `.txt` independientes (``alumnos.txt`` y ``profesores.txt``) para asegurar que la información no se pierda al cerrar el programa.
- **Arquitectura Orientada a Objetos (Jerarquía de Entidades):** Implementación de herencia para separar lógicamente a los actores (Clase base ``Persona`` que deriva en ``Alumno`` y ``Profesor``) y mantener el código escalable.
- **Automatización Polimórfica de Aprobación:** Un sistema que evalúe dinámicamente si un alumno aprueba o reprueba según las reglas específicas de su programa (10/20 para Cursos, 14/20 para Diplomados, y cero notas por debajo de 14 para Bootcamps) sin intervención manual.
- **Estructuras de Datos Lineales para Seguridad y Eficiencia:**
  - Una **Pila (LIFO)** para proteger la integridad de los datos permitiendo deshacer el último registro de nota ingresado por error.
  - Una **Cola (FIFO)** para procesar el flujo de graduandos y generar automáticamente el reporte de salida (``certificados_pendientes.txt``).

- **Interfaz de Consola (Menú de 7 Opciones):** Un bucle continuo que permita registrar usuarios, cargar notas, deshacer errores y emitir reportes de forma intuitiva para el operador administrativo.

### 3. Plan Lógico de Trabajo

El desarrollo del sistema se ejecutará de manera secuencial, asegurando que la base arquitectónica sea lo suficientemente robusta para ser adaptada posteriormente a las filosofías particulares de ejecución y manejo de memoria en Python, Java y C++.

1. **Fase 1: Diseño de Arquitectura y Clases Base**  
Modelado y programación de las clases principales (``Persona``, ``Alumno``, ``Profesor``, ``ProgramaAcademico``). Definición de los atributos y constructores para encapsular correctamente la información.
2. **Fase 2: Implementación de Lógica de Negocio (Polimorfismo)**  
Desarrollo de los métodos de evaluación de notas para las clases derivadas (``Curso``, ``Diplomado``, ``Bootcamp``). Pruebas unitarias de los cálculos de promedios para garantizar que el cuello de botella de las certificaciones quede resuelto.
3. **Fase 3: Desarrollo de Módulos de Persistencia (I/O)**  
Creación de las funciones de lectura y escritura para manejar los archivos ``alumnos.txt`` y ``profesores.txt``. Se asegurará que el formato por comas cumpla estrictamente con el diccionario de datos esperado.
4. **Fase 4: Estructuras de Datos Avanzadas**  
Integración de la Pila (Stack) en memoria para el control de la función "Deshacer Último Registro de Nota" y la implementación de la Cola (Queue) para la generación del reporte de certificados pendientes.
5. **Fase 5: Interfaz de Usuario e Integración Final**  
Construcción del menú interactivo por consola. Conexión de las 7 opciones del menú con las funciones correspondientes del backend. Pruebas de estrés y manejo de excepciones (ej. qué pasa si el archivo ``*.txt`` no existe o el usuario ingresa una opción inválida).